BrainAI

ProSuite Industrial

KERNEL & USERSPACE ENCRYPTION

PRODUKT-DOKUMENTATION

BrainAI (Inhaber: Sascha Alexander Köhne)

BrainAI Inhaber: Sascha Alexander Köhne	PROSUITE INDUSTRIAL	🟢 ÖFFENTLICHE KLASSIFIZIERUNG
Core Architecture & Development	**ProED Engine CLI Referenz**	Zugriffsstufe: Öffentlich / Beta

---

Revisionshistorie / Revision History

Version	Datum	Autor	Status	Beschreibung
1.0.0	01.01.2025	S. Köhne	Stable	Initial Command Line Interface
2.0.0	26.02.2026	S. Köhne	Stable	Hybrid Ring-0 Edition (ProKM Kernel Support & Armored Envelope)

1 Produktübersicht

Die ProED Executable (`proed.exe` / `proed`) ist das hochperformante Command-Line-Interface (CLI) der ProED Core Engine. Sie ermöglicht die nahtlose Verschlüsselung und Entschlüsselung von Dateien und Datenströmen direkt über das Terminal oder durch Automatisierungsskripte.

Kernmerkmale

- **Hybride Beschleunigung:** Die Executable erkennt automatisch, ob der `ProKM` Kernel-Treiber auf dem Host-System installiert ist. Wenn ja, werden große Dateien (z. B. Backups oder Log-Archive) direkt über Ring-0 verarbeitet, andernfalls übernimmt der native User-Space-Fallback.
- **Stream-Processing:** Dateien werden speichereffizient in Chunks verarbeitet. Selbst Terabyte-große Datenbank-Dumps können auf Systemen mit minimalem RAM sicher verschlüsselt werden.
- **Armored Envelope (AEAD):** Die CLI verpackt verschlüsselte Dateien automatisch mit dem 48-Byte Header (inklusive IV und MAC). Bei der Entschlüsselung wird die Datei strikt abgelehnt, falls auch nur ein Bit verändert wurde.
- **Automatisierungsfreundlich:** Strikte POSIX-kompatible Exit-Codes (Return Codes) ermöglichen eine sichere Auswertung in CI/CD-Pipelines und Shell-Skripten.

2. Technische Spezifikationen

2.1 Globale Konstanten

Die Executable hält sich strikt an die Limits und Vorgaben der Core Engine:

- **Envelope Header:** *48* Bytes (Wird bei der Verschlüsselung an den Anfang der Datei geschrieben).
- **Key Size:** Exakt *32* Bytes (*256*-Bit). Kann als Raw-Binärdatei (`-k key.bin`) oder als Hex-String (`--hexkey`) übergeben werden.
- **Chunk Size (I/O):** Standardmäßig 4 MB (Optimiert für Festplatten- und Kernel-I/O).

2.2 Kommandozeilen-Parameter (CLI)

Die Executable erwartet folgende Syntax: `proed [MODUS] [OPTIONEN]`

- `-e, --encrypt` : Startet den Verschlüsselungsmodus.
- `-d, --decrypt` : Startet den Entschlüsselungsmodus.
- `-i, --input <file>` : Pfad zur Eingabedatei.
- `-o, --output <file>`: Pfad zur Ausgabedatei.
- `-k, --key <file>` : Pfad zur 32-Byte Schlüsseldatei.
- `--hexkey <string>` : Alternativ: Der 32-Byte Schlüssel als 64-Zeichen langer Hex-String.

3. Globales Error-Management (Exit Codes)

Für die nahtlose Integration in Skripte nutzt die ProED Executable standardisierte Exit-Codes (Return Values). Ein Wert von `0` bedeutet immer Erfolg.

Exit Code	***Bedeutung***	***Beschreibung***	***Kritikalität***
0	SUCCESS	Operation erfolgreich abgeschlossen. Datei wurde verifiziert und geschrieben.	*Keine*
1	ERR_PARAM	Falsche CLI-Parameter (z.B. fehlende Argumente, ungültige Schlüssellänge).	*Mittel (Logikfehler)*
2	ERR_AUTH	MAC-Vergleich fehlgeschlagen. Die Eingabedatei wurde manipuliert oder ist beschädigt!	*Hoch (Angriff!)*
3	ERR_IO	Lese-/Schreibfehler (z.B. Datei nicht gefunden, keine Schreibrechte).	*Mittel (Systemfehler)*
4	ERR_KERNEL	Der ProKM Treiber hat einen kritischen Hardware-Fehler gemeldet.	*Fatal*

4. Linux / Bash Integration (Server & CI/CD)

Unter Linux und macOS eignet sich die Executable hervorragend für Cronjobs, Backup-Skripte oder den Einsatz in Docker-Containern.

4.1 Implementierungsbeispiel (Backup-Skript)

```bash
#!/bin/bash

INPUT_FILE="/var/log/syslog.archive"
OUTPUT_FILE="/backups/syslog.archive.proed"
KEY_FILE="/etc/proed/master_key.bin"

echo "Starte sichere Verschlüsselung..."

# ProED CLI aufrufen
./proed --encrypt -i "$INPUT_FILE" -o "$OUTPUT_FILE" -k "$KEY_FILE"

# Exit-Code prüfen
EXIT_CODE=$?

if [ $EXIT_CODE -eq 0 ]; then
    echo "Backup erfolgreich verschlüsselt und gesichert."
    rm "$INPUT_FILE" # Originaldatei löschen
elif [ $EXIT_CODE -eq 3 ]; then
    echo "Fehler: Konnte Datei nicht lesen oder schreiben (I/O Error)
else
    echo "Kritischer Fehler bei der Verschlüsselung! (Code: $EXIT_COD
    exit 1
fi
```

5. Windows PowerShell Integration (Industrie & OT)

Auf Windows-Systemen (insbesondere in SCADA- und SPS-Umgebungen) wird die `proed.exe` oft genutzt, um Konfigurationsdateien oder Sensor-Dumps sicher zu verarbeiten, bevor sie über das Netzwerk geschickt werden.

5.1 Implementierungsbeispiel (Sicherer Datei-Import)

```
$EncryptedFile = "C:\OT_Data\sensor_dump.proed"
$DecryptedFile = "C:\OT_Data\sensor_dump.csv"
$KeyHex = "A1B2C3D4E5F6..." # 64-Zeichen Hex-String

Write-Host "Überprüfe und entschlüssele Sensor-Daten..."

# ProED Executable aufrufen und auf Beendigung warten
$process = Start-Process -FilePath "proed.exe" -ArgumentList "-d -i $

# Strikte Auswertung des Return Codes
if ($process.ExitCode -eq 0) {
    Write-Host "Integrität bestätigt. Datei wurde erfolgreich entschl
    # Import in die Steuerungssoftware starten
}
elseif ($process.ExitCode -eq 2) {
    Write-Host "SICHERHEITSALARM: Datei wurde manipuliert (MAC-Fehler
    # Alarm auslösen, Anlage in den sicheren Zustand versetzen
}
else {
    Write-Host "Allgemeiner Systemfehler (Code: $($process.ExitCode))
}
```

6. Python Subprocess Integration

Wenn keine native C-Bindings (`ctypes`) verwendet werden sollen oder können (z.B. in restriktiven Umgebungen), kann die Executable direkt aus Python heraus sicher orchestriert werden.

6.1 Implementierungsbeispiel

```
import subprocess
import os

def decrypt_secure_file(input_path: str, output_path: str, key_path:
    """Entschlüsselt eine Datei via ProED CLI und prüft auf Manipulat

    if not os.path.exists(input_path):
        raise FileNotFoundError(f"Eingabedatei {input_path} fehlt.")

    # Kommando sicher als Liste übergeben (verhindert Shell-Injection
    cmd = [
        "./proed",
        "--decrypt",
        "-i", input_path,
        "-o", output_path,
        "-k", key_path
    ]

    try:
        # Führt die CLI aus und fängt den Exit-Code
        result = subprocess.run(cmd, capture_output=True, text=True,

        if result.returncode == 0:
            print("Erfolg: Datei ist authentisch und entschlüsselt.")
            return True
        elif result.returncode == 2:
            print("CRITICAL: Strikte Abweisung! Die Datei wurde manip
            return False
        else:
            print(f"Engine Fehler (Code {result.returncode}): {result
            return False

    except Exception as e:
```

```
        print(f"Systemfehler beim Aufruf der Executable: {e}")
        return False
```

7. Node.js Child Process Integration

In JavaScript/Node.js-Backends kann die Executable über `child_process` asynchron aufgerufen werden, um rechenintensive Ver- oder Entschlüsselungen von großen Dateien in einen separaten Betriebssystem-Prozess auszulagern (verhindert das Blockieren des Main-Threads).

7.1 Implementierungsbeispiel

```
const { spawn } = require('child_process');

function encryptFileAsync(inputPath, outputPath, hexKey) {
    return new Promise((resolve, reject) => {

        // Asynchroner Aufruf der Executable
        const proed = spawn('./proed', [
            '-e',
            '-i', inputPath,
            '-o', outputPath,
            '--hexkey', hexKey
        ]);

        proed.on('close', (code) => {
            if (code === 0) {
                console.log("Verschlüsselung via CLI erfolgreich abge
                resolve(true);
            } else if (code === 3) {
                reject(new Error("I/O Fehler: Überprüfen Sie die Date
            } else {
                reject(new Error(`Kritischer Engine-Fehler. Exit-Code
            }
        });

        proed.on('error', (err) => {
            reject(new Error(`Fehler beim Starten der Executable: ${e
        });
    });
}
```

8. Best Practices & Sicherheitsrichtlinien

- **Schlüssel-Übergabe:** Bevorzugen Sie in produktiven Skripten immer die Übergabe des Schlüssels als Datei (`-k key.bin`) anstatt per Hex-String (`--hexkey`). Parameter, die über die Kommandozeile (CLI) übergeben werden, sind für andere Benutzer des Systems unter Umständen kurzzeitig in der Prozessliste (z. B. via `htop` oder Task-Manager) sichtbar.
- **Sicheres Löschen (Wiping):** Die Executable löscht den Schlüssel und alle sensiblen Daten nach der Verarbeitung automatisch aus dem eigenen RAM. Das Löschen der unverschlüsselten Originaldatei von der Festplatte obliegt jedoch dem aufrufenden Skript (z.B. durch Tools wie `shred` oder `sdelete`).
- **Kernel-Fallback:** Die Executable erkennt den ProKM Treiber vollautomatisch. Es sind keine gesonderten CLI-Switches nötig. Stellen Sie jedoch sicher, dass der ausführende Benutzer (oder der Service-Account) die nötigen Berechtigungen besitzt, um mit `\\.\ProKM` (Windows) bzw. `/dev/prokm` (Linux) zu kommunizieren.

© 2026 Sascha Köhne	✉️ **Contact:** koehne83@googlemail.com	🛡️ **ProTU-Verified**
CEO / System Architect	BrainAI Inhaber : Sascha Alexander Köhne	We don't need **BRUTEFORCE**, we know **PHYSICS**